

Mission SoC 레지스터 데이터시트

대상 하드웨어 — Basys 3 (XC7A35T-1CPG236C) / MicroBlaze RISC-V / AXI4-Lite / 100 MHz

이 문서는 Custom IP 세 개(myip_heartbeat_monitor, fault_manager_ip, safety_controller)의 레지스터를 완전 사양으로 다루고, 보조 RTL eval_tick_generator 와 표준 IP(axi_gpio, axi_intc, axi_uartlite)는 이 설계에서 실제로 쓰는 연결과 레지스터만 기술합니다. 표준 IP 의 전체 레지스터 사양 은 AMD 벤더 문서(PG144 / PG099 / PG142)를 참고하십시오.

1. 시스템 주소 맵

MicroBlaze RISC-V Data 주소 공간 기준입니다. 근거 — SOC_Pr/soc_project/soc_project.srcs/sources_1/bd/mission_soc/mission_soc.bd

Base	Range	인스턴스	VLNV	구분
0x0000_0000	128 K	dlmb_bram_if_cntlr	LMB BRAM	범위 밖
0x4000_0000	64 K	axi_gpio_0	xilinx.com:ip:axi_gpio:2.0	표준 IP
0x4001_0000	64 K	axi_gpio_1	xilinx.com:ip:axi_gpio:2.0	표준 IP
0x4060_0000	64 K	axi_uartlite_0	xilinx.com:ip:axi_uartlite:2.0	표준 IP
0x4120_0000	64 K	microblaze_riscv_0_axi_intc	xilinx.com:ip:axi_intc:4.1	표준 IP
0x44A0_0000	64 K	fault_manager_ip_0	user.org:user:fault_manager_ip:1.0	Custom IP
0x44A1_0000	64 K	myip_heartbeat_monit_0	user.org:user:myip_heartbeat_monitor:1.0	Custom IP
0x44A2_0000	64 K	safety_controller_0	user.org:user:safety_controller:1.0	Custom IP

Instruction 주소 공간은 ilmb_bram_if_cntlr 128 K 하나뿐입니다.

펌웨어는 이 주소를 직접 쓰지 않고 mission_ip_regs.h 의 FM_BASE / HB_BASE / SC_BASE 매크로를 통해 xparameters.h 값을 참조합니다.

2. Custom IP 공통 AXI4-Lite 규약

2.1 세 IP 공통

항목	값
프로토콜	AXI4-Lite Slave
Data Width	32 bit 고정
지원 전송	32 bit 정렬 단일 Read / Write
BRESP / RRESP	항상 OKAY(2'b00)
Outstanding	미지원. 응답을 받은 뒤 다음 트랜잭션을 발행합니다
AW/W 채널 순서	독립 수신 후 Commit. 어느 쪽이 먼저 도착해도 정상 동작합니다
Read-only 레지스터 Write	무시. 오류 응답 없음
미정의 오프셋 Read	0x0000_0000
미정의 오프셋 Write	무시
Reset	S_AXI_ARESETN Active-Low. Core 에는 반전해 Active-High 로 전달합니다

SLVERR 가 없습니다. 세 IP 모두 어떤 주소, 어떤 접근에도 OKAY 를 반환합니다. 오타로 잘못된 오프셋을 읽으면 예외 없이 0 이 돌아오고, 잘못된 오프셋에 쓰면 조용히 버려집니다. 브링업에서 주소 매핑을 확인하려면 fault_manager_ip 의 ID(0x2C) 레지스터를 읽으십시오.

2.2 IP 별로 다른 항목

항목	heartbeat_monitor	fault_manager	safety_controller
C_S_AXI_ADDR_WIDTH	6	6	5
주소 디코딩 방식	ADDR[5:0] 완전 비교	ADDR[5:2] (word index)	ADDR[4:0] 완전 비교
에일리어싱 주기	64 B	64 B	32 B
비정렬 주소 (+1,+2,+3)	미정의 → 0	하위 2비트 무시 → 정렬 주소와 동일	미정의 → 0
WSTRB 처리	바이트 단위 반영	WSTRB[0]==0 이면 Write 전체 무시	바이트 단위 반영

항목	heartbeat_monitor	fault_manager	safety_controller
IRQ_EN / IRQ_STATUS 오프셋	0x20 / 0x24	0x20 / 0x24	0x18 / 0x1C

⚠ 에일리어싱 주의

Address Editor 는 IP 당 64 KB 를 할당하지만 IP 포트는 위 표의 폭만 받습니다. 상위 주소 비트는 AXI Interconnect 에서 잘려 IP 에 도달하지 않습니다. 따라서 aperture 안에서 레지스터 블록이 반복해서 노출됩니다.

접근 주소	실제로 닿는 레지스터
HB_BASE + 0x40	HB_BASE + 0x00 (CTRL)
FM_BASE + 0x40	FM_BASE + 0x00 (CTRL)
FM_BASE + 0x02	FM_BASE + 0x00 (CTRL)
SC_BASE + 0x20	SC_BASE + 0x00 (CTRL)

마지막 줄이 실질적인 사고 지점입니다. `safety_controller` 의 `IRQ_EN` 은 0x18 인데 `fault_manager` 와 `heartbeat_monitor` 는 0x20 입니다. 습관대로 SC 에 0x20 을 쓰면 CTRL 에 써지면서 `ENABLE` 이 `WDATA[0]` 으로 덮어써집니다. IP 가 조용히 꺼지고 출력이 전부 차단되며, 오류 응답은 나오지 않습니다.

반드시 `mission_ip_regs.h` 의 `SC_IRQ_EN / SC_IRQ_STATUS` 매크로를 사용하십시오.

2.3 CTRL 레지스터의 W1P 규약

세 IP 모두 CTRL 이 RW(ENABLE) 과 W1P(펄스 명령) 을 한 레지스터에 함께 두고 있습니다. 그리고 CTRL Write 가 일어날 때마다 ENABLE 이 WDATA[0] 값으로 무조건 덮어씌웁니다.

IP	근거
fault_manager	fault_manager_axi.v:207 — reg_enable <= wdata_hold[0];
safety_controller	safety_controller_slave_lite_v1_0_S00_AXI.v:238 — reg_enable <= wdata_hold[0];
heartbeat_monitor	heartbeat_monitor.v:184 — ctrl_reg <= apply_wstrb32(...) & 32'h5;

따라서 W1P 명령만 단독으로 보내면 IP 가 꺼집니다.

```
/* 틀린 사용 - ENABLE 이 0 으로 덮어써져 IP 가 꺼짐 */
REG_WR(FM_BASE, FM_CTRL, FM_CTRL_RESET_FAULT);

/* 올바른 사용 - CTRL Shadow 에 ENABLE 을 실어 함께 전송 */
REG_WR(FM_BASE, FM_CTRL, s_fm_ctrl | FM_CTRL_RESET_FAULT);
```

fm_regs.c / hb_regs.c / sc_regs.c 가 각각 CTRL Shadow 를 유지하는 이유입니다. 드라이버를 우회해 레지스터를 직접 쓰지 마십시오.

W1P 비트는 레지스터에 저장되지 않습니다. Read 하면 항상 0 입니다.

2.4 IRQ 규약

- **Level 방식**입니다. `irq = |(IRQ_STATUS & IRQ_EN)` 이므로, ISR 이 W1C 로 Pending 을 지울 때까지 High 를 유지합니다.
- `IRQ_STATUS` 는 **W1C**(Write-1-to-Clear)입니다. 1 을 쓴 비트만 지워지고, 0 을 쓴 비트는 그대로 남습니다.
- **Event Set** 이 **W1C Clear** 보다 **우선합니다**. 같은 클럭에 새 이벤트와 Clear 가 겹쳐도 Pending 을 잃지 않습니다.
- IP 가 `ENABLE=0` 이면 새 Pending 이 생기지 않습니다. 다만 이미 쌓인 Pending 은 남아 있습니다.

ISR 표준 절차입니다.

1. IRQ_STATUS 읽기
 2. 읽은 가스를 그대로 IRQ_STATUS 에 되쓰기 (W1C)
 3. 상태 레지스터 스냅샷 저장
 4. 무거운 처리는 메인 루프로 미루기

3. myip_heartbeat_monitor (Base 0x44A1_0000)

3.1 개요

장치 세 개의 비동기 Heartbeat 입력을 2FF 로 동기화한 뒤 상승 에지로 검출하고, 장치별 Timeout Counter 와 `alive / timeout / Timeout Event` 를 생성합니다.

항목	값
Top module	myip_heartbeat_monitor

항목	값
계층	myip_heartbeat_monitor → heartbeat_monitor_axi → heartbeat_monitor_core → heartbeat_monitor_channel × 3
사양 근거	SOC_Pr/ip_repo/myip_heartbeat_monitor_1_0/src/heartbeat_monitor.v
Aperture	64 KB 할당 / 64 B 주기 에일리어싱

hdl/myip_heartbeat_monitor_slave_lite_v1_0_S00_AXI.v를 사양 근거로 쓰지 마십시오.

이 파일은 component.xml의 fileSet에 포함되어 있지 않습니다. 합성에 들어가지 않는 폐기된 Vivado 마법사 스켈레톤입니다. 실제 경로는 hdl/myip_heartbeat_monitor.v가 src/heartbeat_monitor.v의 heartbeat_monitor_axi를 인스턴스하는 쪽입니다.

두 파일은 동작이 다릅니다. 폐기된 파일은 slv_reg10~15가 살아 있어 0x28~0x3C가 읽고 쓸 수 있는 스크래치 레지스터로 동작하고, LAST_COUNT 영역에 쓴 값이 내부 레지스터에 남습니다. 실제 IP에는 그런 저장소가 없습니다.

비 AXI 포트

포트	방향	폭	연결
heartbeat_async	in	3	axi_gpio_1 CH1
alive	out	3	LED (led_concat)
timeout	out	3	fault_manager_ip.timeout
irq	out	1	xlconcat In2 → axi_intc ID 2

alive는 fault_manager_ip에 연결하지 않습니다. 04 체크리스트 1.1의 금지 연결입니다.

3.2 레지스터 요약

Offset	이름	접근	리셋값	폭	의미
0x00	CTRL	RW/W1P	0x0000_0000	3 bit 사용	동작 제어
0x04	STATUS	RO	—	6 bit 사용	alive / timeout 현재값
0x08	TIMEOUT0	RW	0x0000_0000	32	Device 0 Timeout 임계 (clocks)
0x0C	TIMEOUT1	RW	0x0000_0000	32	Device 1
0x10	TIMEOUT2	RW	0x0000_0000	32	Device 2
0x14	LAST_COUNT0	RO	—	32	Device 0 마지막 Heartbeat 이후 경과 clocks
0x18	LAST_COUNT1	RO	—	32	Device 1
0x1C	LAST_COUNT2	RO	—	32	Device 2
0x20	IRQ_EN	RW	0x0000_0000	3	Timeout IRQ Enable
0x24	IRQ_STATUS	RO/W1C	0x0000_0000	3	Timeout Pending

0x28 이상은 미정의입니다. Read는 0, Write는 무시됩니다. 0x40 이상은 0x00의 에일리어스입니다.

3.3 0x00 CTRL

Bit	이름	접근	리셋	설명
0	ENABLE	RW	0	0이면 모든 채널의 Counter와 timeout을 0으로 강제하고 alive=0이 됩니다
1	CLEAR_ALL	W1P	—	1을 쓴 클럭에 1클럭 Pulse를 만듭니다. 레지스터에 저장되지 않으며 Read는 항상 0입니다
2	AUTO_RECOVER	RW	0	1이면 Heartbeat 수신 시 timeout 래치를 해제합니다. 0이면 CLEAR_ALL이나 ENABLE=0까지 유지됩니다. 운용값은 1입니다
31:3	—	RO	0	0

저장 마스크가 32'h0000_0005이므로 bit1은 저장 자체가 되지 않습니다 (heartbeat_monitor.v:187).

CLEAR_ALL은 IRQ_STATUS를 지우지 않습니다. Counter와 timeout 래치만 지웁니다 (heartbeat_monitor.v:546-550). Pending을 지우려면 IRQ_STATUS(0x24)에 별도로 W1C를 수행해야 합니다.

3.4 0x04 STATUS (RO)

Bit	이름	설명
2:0	ALIVE[2:0]	장치별 생존 상태. alive[i] = ENABLE && !timeout[i]

Bit	이름	설명
7:3	—	0
10:8	TIMEOUT[2:0]	장치별 Timeout 래치
31:11	—	0

3.5 0x08 / 0x0C / 0x10 TIMEOUTn (RW, 32 bit)

마지막 Heartbeat 이후 이 clocks 수만큼 Heartbeat 가 없으면 timeout[n] 이 Set 됩니다.

- 단위는 100 MHz clock count 입니다. mission_ip_regs.h 의 MS_TO_CLK(ms) 로 변환합니다.
- 0 을 쓰면 RTL 이 1 로 간주합니다 (heartbeat_monitor.v:512-514). 감시를 끄는 수단이 아닙니다. 감시를 멈추려면 CTRL.ENABLE=0 을 사용하십시오.
- 리셋값이 0 이므로 MicroBlaze 가 값을 써 주기 전까지는 1클럭 만에 Timeout 이 성립합니다. 부팅 시 반드시 설정을 마친 뒤 ENABLE 을 켜십시오.

권장 초기값입니다.

Device	용도	ms	clocks
0	일반 센서	300	30,000,000
1	통신	600	60,000,000
2	모터 / 핵심 제어	150	15,000,000

3.6 0x14 / 0x18 / 0x1C LAST_COUNTn (RO, 32 bit)

마지막 Heartbeat 상승 에지 이후 경과한 clock 수입니다.

- Heartbeat 를 수신한 클럭에 0 으로 초기화됩니다.
- 0xFFFF_FFFF 에서 포화하며 Wrap around 하지 않습니다 (heartbeat_monitor.v:524-526).
- ENABLE=0 또는 CLEAR_ALL 시 0 이 됩니다.
- 하드웨어가 세기 때문에 소프트웨어가 멈춰 있어도 정확합니다. hb_gen.c 가 Heartbeat 생성 주기의 시간 기준으로 이 값을 사용합니다.

3.7 0x20 IRQ_EN / 0x24 IRQ_STATUS

Bit	의미
2:0	Device 0 / 1 / 2 의 Timeout IRQ
31:3	0

- IRQ_STATUS 는 timeout 이 0 → 1 로 바뀌는 1클럭 Event 를 래치합니다.
- 갱신식은 irq_status <= (irq_status & ~wlc_mask) | timeout_event 입니다 (heartbeat_monitor.v:269-270). Set 이 Clear 보다 우선합니다.
- irq = |(irq_status & irq_en) 입니다 (heartbeat_monitor.v:274).
- IRQ_STATUS 는 별도 레지스터이므로 CTRL.CLEAR_ALL 과 독립입니다. 3.3 절을 참고하십시오.

3.8 동작 세부

Heartbeat 검출 — 2FF 동기화(ASYNC_REG=TRUE) 후 상승 에지를 검출합니다. 비동기 입력이므로 최소 2 클럭의 지연이 있습니다.

Timeout 판정 우선순위 (heartbeat_monitor.v:539-570)

```
reset
> ENABLE=0 또는 device_enable=0 → counter=0, timeout=0
> CLEAR_ALL → counter=0, timeout=0
> heartbeat 수신 → counter=0, (AUTO_RECOVER 면 timeout=0)
> timeout set → counter_incremented >= timeout_effective 인 클럭
> counter 증가 → 포화 증가
```

device_enable 은 3'b111 로 고정되어 있습니다 (heartbeat_monitor.v:245). 장치별 감시 on/off 레지스터는 없습니다.

4. fault_manager_ip (Base 0x44A0_0000)

4.1 개요

timeout/error_flag/critical_fault 를 받아 지속성과 우선순위로 고장 등급을 산출합니다.

항목	값
Top module	fault_manager_ip
계층	fault_manager_ip → fault_manager_ip_slave_lite_v1_0_S00_AXI (얇은 래퍼) → fault_manager_axi → fault_manager_core
사양 근거	SOC_Pr/ip_repo/fault_manager_ip_1_0/src/fault_manager_axi.v,src/fault_manager_core.v
Aperture	64 KB 할당 / 64 B 주기 에일리어싱

hdl/fault_manager_ip_slave_lite_v1_0_S00_AXI.v 는 마법사 로직을 걷어내고 fault_manager_axi 를 인스턴스만 하는 래퍼입니다. heartbeat IP 의 동명 파일과 달리 이 파일은 실제로 사용됩니다.

비 AXI 포트

포트	방향	폭	연결
timeout	in	3	myip_heartbeat_monitor.timeout
error_flag	in	3	axi_gpio_0 CH1
critical_fault	in	3	axi_gpio_0 CH2
eval_tick	in	1	eval_tick_generator.eval_tick
fault_level/fault_device/fault_code/fault_valid	out	2/2/8/1	safety_controller
irq	out	1	xlconcat In1 → axi_intc ID 1

4.2 레지스터 요약

Offset	이름	접근	리셋값	의미
0x00	CTRL	RW / W1P	0x0000_0000	동작 제어
0x04	FAULT_INPUT	RO	—	세 입력의 현재값
0x08	CRITICAL_MASK	RW	0x0000_0004	임무 필수 장치 지정
0x0C	PERSIST_LIMIT	RW	0x0000_0005	지속 판정 횟수
0x10	FAULT_LEVEL	RO	—	고장 등급 0~3
0x14	FAULT_DEVICE	RO	—	주요 고장 장치 ID
0x18	FAULT_CODE	RO	—	고장 코드
0x1C	FAULT_COUNT	RO	—	장치별 지속 Count
0x20	IRQ_EN	RW	0x0000_0000	Fault 변화 IRQ Enable
0x24	IRQ_STATUS	RO / W1C	0x0000_0000	Fault 변화 Pending
0x28	—	—	—	미구현. 향후 확장용으로 비워 두었습니다
0x2C	ID	RO	0x464D_4752	"FMGR" 상수. 브링업 진단용

이 IP 는 wSTRB[0]==0 인 Write 를 통째로 무시합니다 (fault_manager_axi.v:204). RW 필드가 모두 byte0 안에 있어서 내린 결정입니다. Byte1~3 만 활성화한 Write 는 아무 효과가 없습니다.

4.3 0x00 CTRL

Bit	이름	접근	리셋	설명
0	ENABLE	RW	0	0 이면 Count 를 0 으로 강제하고 출력을 안전값(LEVEL 0 / DEVICE 3 / CODE 0x00)으로 고정합니다. Event 도 만들지 않습니다
1	RESET_FAULT	W1P	—	1클럭 Pulse 를 만듭니다. Read 는 항상 0 입니다
31:2	—	RO	0	0

RESET_FAULT 는 조건부입니다. device_fault == 3'b000 일 때만 지속 Count 와 IRQ_STATUS 를 지웁니다 (fault_manager_axi.v:229, fault_manager_core.v:104). 활성 Fault 가 남아 있으면 명령 자체가 무시됩니다. Count 만 지워 LEVEL 2 를 LEVEL 1 로 낮추는 동작은 금지되어 있습니다. 오류 응답이 없으므로 소프트웨어가 결과를 다시 읽어 확인해야 합니다.

4.4 0x04 FAULT_INPUT (RO)

Bit	이름	출처
2:0	timeout[2:0]	myip_heartbeat_monitor
7:3	—	0
10:8	error_flag[2:0]	axi_gpio_0 CH1
15:11	—	0
18:16	critical_fault[2:0]	axi_gpio_0 CH2
31:19	—	0

주입한 값이 실제로 IP 에 도달했는지 확인하는 용도로 사용합니다. GPIO 출력 레지스터를 되읽는 것보다 이 방법이 확실합니다.

4.5 0x08 CRITICAL_MASK (RW, bit[2:0], 리셋 3'b100)

1 인 장치는 임무 필수 장치로 취급됩니다. 그 장치에 어떤 종류든 Fault 가 잡히면 지속시간 판정을 건너뛰고 즉시 LEVEL 3 SAFE, FAULT_CRITICAL 이 됩니다.

```
crit_active = (timeout | error_flag | critical_fault) & critical_mask;
```

Mask 밖 장치의 critical_fault 는 Critical 로 승격되지 않고 FAULT_ERROR_CODE 로 취급됩니다 (fault_manager_core.v:85).

기본값 0x4 는 Device 2(모터/핵심 제어)만 임무 필수로 지정합니다.

4.6 0x0C PERSIST_LIMIT (RW, bit[7:0], 리셋 5)

eval_tick 기준으로 이 횟수만큼 Fault 가 연속 유지되면 LEVEL 1 WARNING 에서 LEVEL 2 DEGRADED 로 승격됩니다.

- 0 을 쓰면 1 로 간주합니다 (fault_manager_core.v:88).
- 최댓값은 255 입니다. eval_tick 기본 주기가 1 ms 이므로 255 는 약 255 ms 입니다.
- FAULT_COUNT 가 0xFF 에서 포화하므로 PERSIST_LIMIT 을 255 로 두면 DEGRADED → WARNING 하강 판정이 성립하지 않습니다. 운용값은 5 를 사용합니다.

4.7 0x10 / 0x14 / 0x18 판정 결과 (RO)

Offset	이름	필드	인코딩
0x10	FAULT_LEVEL	bit[1:0]	0 NORMAL / 1 WARNING / 2 DEGRADED / 3 SAFE
0x14	FAULT_DEVICE	bit[1:0]	0 Device 0 / 1 Device 1 / 2 Device 2 / 3 MULTIPLE_OR_NONE
0x18	FAULT_CODE	bit[7:0]	0x00 NONE / 0x01 TIMEOUT / 0x02 ERROR_CODE / 0x03 CRITICAL / 0x04 MULTI_DEVICE

0x05 FAULT_RECOVERY_REQ 는 mission_ip_regs.h 에 상수로 정의되어 있으나 fault_manager_core 는 출력하지 않습니다.

판정 우선순위 (fault_manager_core.v:182-207) — 위가 아래를 덮습니다.

우선순위	조건	LEVEL	CODE	DEVICE
P1	crit_active != 0	3 SAFE	0x03 CRITICAL	Critical 장치가 하나면 그 ID, 아니면 3
P2	오류 장치 두 개 이상	3 SAFE	0x04 MULTI_DEVICE	3
P3	지속 조건 성립	2 DEGRADED	단일 장치 코드	해당 장치
P4	오류가 있으나 일시적	1 WARNING	단일 장치 코드	해당 장치
—	그 외	0 NORMAL	0x00 NONE	3

단일 장치 코드는 그 장치에 error_flag 나 critical_fault 가 있으면 0x02 ERROR_CODE, timeout 만 있으면 0x01 TIMEOUT 입니다.

4.8 0x1C FAULT_COUNT (RO)

Bit	내용
7:0	Device 0 지속 Count
15:8	Device 1
23:16	Device 2
31:24	0

- eval_tick 인 클럭에만 갱신됩니다.
- 해당 장치에 Fault 가 있으면 +1, 없으면 0 이 됩니다.
- 0xFF 에서 포함합니다 (fault_manager_core.v:73, 119).
- ENABLE=0 이면 0 입니다.

4.9 0x20 IRQ_EN / 0x24 IRQ_STATUS

Bit	의미
0	Fault Change (<i>fault_level</i> / <i>fault_device</i> / <i>fault_code</i> 중 하나라도 변경)
31:1	0

- Event 는 세 출력의 묶음 비교로 만듭니다. 같은 값이 유지되는 동안에는 반복해서 Set 하지 않습니다 (*fault_manager_core.v:216*).
- *IRQ_STATUS* 갱신 순서상 Set 이 W1C 와 *RESET_FAULT* 보다 우선합니다 (*fault_manager_axi.v:229-233*).
- *ENABLE=0* 이면 새 Event 가 생기지 않습니다.

4.10 0x2C ID (RO, 0x464D_4752)

ASCII "FMGR" 상수입니다. 원래 확정 맵에 없던 확장이며 2026-07-30 CHANGE REQUEST 로 승인되었습니다. Fault 정책이나 IRQ, 상태 어디에도 영향을 주지 않습니다. 부팅 시 *FM_SelfCheck()* 가 이 값을 읽어 AXI 주소 매핑이 붙었는지 확인합니다. *SLVERR* 가 없는 설계이므로 이 레지스터가 사실상 유일한 주소 매핑 검증 수단입니다.

5. safety_controller (Base 0x44A2_0000)

5.1 개요

fault_level 을 받아 시스템 상태를 전이시키고 장치별 출력을 차단합니다. *SAFE_MODE* 는 래치이며 조건부 수동 승인으로만 해제됩니다.

항목	값
Top module	<i>safety_controller_axi</i> (<i>hdl/safety_controller_slave_lite_v1_0_S00_AXI.v</i>)
계층	<i>safety_controller_axi</i> → <i>safety_controller_core</i> (<i>hdl/safety_controller.v</i>)
Aperture	64 KB 할당 / 32 B 주기 에일리어싱 (다른 두 IP 와 다릅니다)

비 AXI 포트

포트	방향	폭	연결
<i>fault_level</i> / <i>fault_device</i> / <i>fault_code</i> / <i>fault_valid</i>	in	2/2/8/1	<i>fault_manager_ip</i>
<i>eval_tick</i>	in	1	<i>eval_tick_generator.eval_tick</i>
<i>output_enable</i>	out	3	LED (<i>led_concat</i>)
<i>actuator_enable</i>	out	1	LED. AXI 로 읽을 수 없습니다
<i>control_valid</i>	out	1	LED. AXI 로 읽을 수 없습니다
<i>irq</i>	out	1	<i>xlconcat</i> In3 → <i>axi_intc</i> ID 3

5.2 레지스터 요약

Offset	이름	접근	리셋값	의미
0x00	CTRL	RW / W1P	0x0000_0000	동작 제어
0x04	SYSTEM_STATE	RO	—	현재 시스템 상태
0x08	OUTPUT_ENABLE	RO	—	장치별 출력 허용
0x0C	DEGRADE_MASK	RW	0x0000_0000	DEGRADED 이면서 장치 특정 불가일 때 차단할 출력
0x10	RECOVERY_COUNT	RW	0x0000_0000	하강 전이에 필요한 연속 정상 횟수
0x14	STATE_TIMER	RO	—	현재 상태 유지 clocks
0x18	IRQ_EN	RW	0x0000_0000	상태 변화 IRQ Enable
0x1C	IRQ_STATUS	RO / W1C	0x0000_0000	상태 변화 Pending

IRQ_EN 과 *IRQ_STATUS* 오프셋이 다른 두 IP 와 다릅니다. FM 과 HB 는 0x20 / 0x24, SC 는 0x18 / 0x1C 입니다. 그리고 SC 의 디코딩 폭이 5비트이므로 0x20 은 0x00 CTRL 의 에일리어스입니다. 2.2 절의 주의 사항을 반드시 확인하십시오.

5.3 0x00 CTRL

Bit	이름	접근	리셋	설명
0	ENABLE	RW	0	0 이면 상태를 NORMAL 로, STATE_TIMER 를 0 으로 되돌리고 모든 출력을 차단합니다. Event 도 만들지 않습니다
1	MANUAL_RESET	W1P	—	1클럭 Pulse 를 만듭니다. Read 는 항상 0 입니다

Bit	이름	접근	리셋	설명
31:2	—	RO	0	0

MANUAL_RESET 은 세 조건이 모두 맞아야 인정됩니다 (`safety_controller.v:104-113`).

```
current_state == SAFE_MODE && fault_valid == 1 && fault_level == 0
```

하나라도 어긋나면 무시됩니다. AXI 응답은 **OKAY** 이므로 소프트웨어가 **SYSTEM_STATE** 를 다시 읽어 승인 여부를 확인해야 합니다. 펌웨어는 이때 **\$ERR, MANUAL_RESET, FAULT_ACTIVE** 를 내보냅니다.

fault_valid 는 **fault_manager_ip** 의 **ENABLE** 과 같습니다. FM 이 꺼져 있으면 SC 도 복구되지 않습니다.

5.4 0x04 **SYSTEM_STATE** (RO, bit[1:0])

값	상태	의미
0	NORMAL	정상
1	WARNING	일시 오류 감지. 출력은 유지
2	DEGRADED	지속 오류. 해당 장치만 차단
3	SAFE_MODE	전면 차단. 래치

전이 규칙 (`safety_controller.v:88-276`)

- 상승(악화)은 **eval_tick** 을 기다리지 않고 다음 클럭에 즉시 반영됩니다.
- 하강(복구)은 **eval_tick** 에서만 Count 가 올라가며, **RECOVERY_COUNT** 회 연속 확인이 필요합니다.
- 하강 경로는 **WARNING→NORMAL**, **DEGRADED→WARNING**, **DEGRADED→NORMAL** 세 가지뿐입니다.
- SAFE_MODE** 는 자동 복구가 없습니다. **MANUAL_RESET** 전용 경로입니다.
- 정의되지 않은 상태값은 안전을 위해 **SAFE_MODE** 로 보냅니다.

5.5 0x08 **OUTPUT_ENABLE** (RO, bit[2:0])

1 인 비트의 장치 출력이 허용됩니다. 조합 논리이므로 상태와 같은 클럭에 바뀝니다.

조건	OUTPUT_ENABLE
ENABLE=0 또는 fault_valid=0	3'b000
NORMAL	3'b111
WARNING	3'b111
DEGRADED , fault_device=0	3'b110
DEGRADED , fault_device=1	3'b101
DEGRADED , fault_device=2	3'b011
DEGRADED , fault_device=3	3'b111 & ~DEGRADE_MASK
SAFE_MODE	3'b000

5.6 0x0C **DEGRADE_MASK** (RW, bit[2:0], 리셋 0)

DEGRADED 상태에서 **fault_device == 3**(다중 장치 또는 특정 불가)일 때만 사용됩니다. 1 인 비트의 출력을 차단합니다.

리셋값이 0 이므로 설정하지 않으면 이 경우에 아무것도 차단하지 않습니다. 권장 초기값은 0x1 입니다.

5.7 0x10 **RECOVERY_COUNT** (RW, bit[15:0], 리셋 0)

하강 전이에 필요한 **eval_tick** 연속 확인 횟수입니다.

- 0 을 쓰면 1 로 간주합니다 (`safety_controller.v:71-74`).
- 상승 전이에는 전혀 관여하지 않습니다.
- SAFE_MODE** 탈출과도 무관합니다. **MANUAL_RESET** 전용 경로이기 때문입니다.
- WSTRB [1:0]** 을 모두 반영하므로 16 bit 전체를 바이트 단위로 쓸 수 있습니다.
- 권장값은 2 이며, **RECOVERY_COUNT < PERSIST_LIMIT** 조건을 지켜야 합니다.

5.8 0x14 **STATE_TIMER** (RO, 32 bit)

현재 상태를 유지한 clock 수입니다.

- 단위는 100 MHz clock count 입니다. **CLK_TO_MS()** 로 ms 변환합니다.

- 상태가 바뀌는 클럭에 0 으로 초기화됩니다.
- 0xFFFF_FFFF 에서 포화합니다 (safety_controller.v:315-317). 약 42.9 초에 해당합니다.
- ENABLE=0 이면 0 입니다.

5.9 0x18 IRQ_EN / 0x1C IRQ_STATUS

Bit	의미
0	상태 변화 (current_state != next_state)
31:1	0

- Event 는 상태가 실제로 바뀌는 클럭에만 1클럭 발생합니다.
- Set 이 W1C 보다 우선합니다 (safety_controller_slave_lite_v1_0_S00_AXI.v:279-287).
- ENABLE=0 이면 Event 를 만들지 않습니다.

SAFE_MODE 래치를 확인할 때 이 레지스터가 근거가 됩니다. 고장 원인을 모두 제거해 FAULT_LEVEL 이 0 이 되어도 상태는 SAFE_MODE 그대로이므로 IRQ_STATUS 에 새 Pending 이 생기지 않습니다.

6. eval_tick_generator (AXI 레지스터 없음)

fault_manager_ip 와 safety_controller 가 같은 시간 단위를 쓰도록 1클럭 폭 Pulse 를 생성합니다. Block Design 에 Module Reference 로 추가한 공통 보조 RTL 이며 Custom IP 로 패키징되어 있지 않습니다.

항목	값
VLNV	xilinx.com:module_ref:eval_tick_generator:1.0 (Module Reference)
사양 근거	rtl/eval_tick_generator.v
파라미터	DIVISOR (integer, 기본 100_000)
포트	clk, reset (active high), eval_tick(out)
기본 주기	100 MHz 기준 1 ms
Counter 폭	32 bit

타이밍 계약

- reset=1 동안 eval_tick=0 이고 내부 Divider 는 0 입니다.
- Reset 해제 후 DIVISOR 클럭을 센 뒤 첫 Pulse 가 나옵니다.
- Pulse 폭은 정확히 1클럭입니다.
- Testbench 는 DIVISOR 만 작은 값으로 Override 합니다. RTL 을 수정하지 않습니다.

이 주기가 바뀌면 FM_PERSIST_LIMIT 과 SC_RECOVERY_COUNT 의 실제 시간 의미가 함께 바뀝니다. 두 레지스터는 시간이 아니라 Tick 횟수를 셉니다.

7. 표준 IP 연결 사양

레지스터 전체 사양은 각 벤더 문서가 정본입니다. 여기에는 이 설계가 실제로 사용하는 부분만 기술합니다.

7.1 axi_gpio_0 — Fault 주입 (Base 0x4000_0000)

xilinx.com:ip:axi_gpio:2.0 · 벤더 문서 PG144

파라미터		값
C_IS_DUAL		1 (2 채널)
C_GPIO_WIDTH / C_GPIO2_WIDTH		3 / 3
C_ALL_OUTPUTS / C_ALL_OUTPUTS_2		1 / 1
Offset	레지스터	이 설계에서의 의미
0x00	GPIO_DATA (CH1)	bit[2:0] → fault_manager_ip.error_flag[2:0]
0x08	GPIO2_DATA (CH2)	bit[2:0] → fault_manager_ip.critical_fault[2:0]

GPIO_TRI(0x04) 와 GPIO2_TRI(0x0C) 는 설정할 필요가 없습니다. 두 채널 모두 C_ALL_OUTPUTS=1 이라 방향이 출력으로 고정되어 있습니다.
주입 결과는 GPIO 데이터 레지스터를 뒤읽지 말고 fault_manager_ip 의 FAULT_INPUT(0x04) 으로 확인하십시오. 실제로 IP 에 도달했는지를 보는 것이 목적인
니다.

7.2 axi_gpio_1 — Heartbeat 주입 (Base 0x4001_0000)

xilinx.com:ip:axi_gpio:2.0 · 벤더 문서 PG144

파라미터	값	
C_IS_DUAL	0 (단일 채널)	
C_GPIO_WIDTH	3	
C_ALL_OUTPUTS	1	
Offset	레지스터	이 설계에서의 의미
0x00	GPIO_DATA (CH1)	bit[2:0] → myip_heartbeat_monitor.heartbeat_async[2:0]

용도가 바뀐 이력이 있습니다. 예전에는 `timeout` 을 직접 주입하는 포트였고, A 의 IP 가 통합되면서 04 체크리스트 1.1 Freeze 에 따라 Heartbeat 주입으로 바뀌었습니다. 지금 `timeout` 은 `myip_heartbeat_monitor` 가 스스로 만듭니다. 오래된 코드를 참고할 때 주의하십시오.

`heartbeat_monitor` 는 상승 에지를 검출합니다. 값을 High 로 유지하는 것은 Heartbeat 가 아닙니다. `hb_gen.c` 가 주기적으로 0 → 1 → 0 을 토글합니다.

7.3 axi_intc — 인터럽트 (Base 0x4120_0000)

xilinx.com:ip:axi_intc:4.1 · 벤더 문서 PG099 · C_HAS_FAST=1

입력은 `microblaze_riscv_0_xlconcat(NUM_PORTS=4)` 이 모음니다. 이 연결 순서가 그대로 인터럽트 ID 이며, 04 체크리스트 8장에 따라 변경할 수 없습니다.

xlconcat 입력	소스	Interrupt ID	mission_intr.h
In0	axi_uartlite_0.interrupt	0	INTR_ID_UART
In1	fault_manager_ip_0 irq	1	INTR_ID_FM
In2	myip_heartbeat_monit_0.irq	2	INTR_ID_HB
In3	safety_controller_0.irq	3	INTR_ID_SC

Custom IP 세 개의 `irq` 는 모두 Level 방식입니다. INTC 를 Edge 로 설정하면 Pending 이 유실됩니다. 레지스터 조작은 BSP 의 `XIntc` 드라이버를 사용합니다.

7.4 axi_uartlite — GUI 링크 (Base 0x4060_0000)

xilinx.com:ip:axi_uartlite:2.0 · 벤더 문서 PG142

항목	값
Board Interface	usb_uart (Basys 3 USB-UART Bridge)
통신 설정	9600 baud, 8-N-1
Interrupt	ID 0

프로토콜(\$MISSION / \$EVENT / \$ACK / \$ERR)은 `uart_proto.c` 가 담당합니다.

TX 는 반드시 `PROTO_Printf()` 를 사용하십시오. `xil_printf` 는 블로킹이라 루프 안에서 쓰면 RX 를 펌프하지 못해 GUI 명령이 유실됩니다.

7.5 범위 밖 IP

`microblaze_riscv_0`, `microblaze_riscv_0_local_memory`(LMB BRAM), `clk_wiz`, `proc_sys_reset_0`, `microblaze_riscv_0_axi_periph`(AXI Interconnect), `mdm_1`, `led_concat`.

레지스터 관점에서 이 설계가 직접 프로그래밍하지 않습니다. `clk_wiz` 는 `sys_clock` 을 받아 100 MHz 를 만들며, 이 문서의 모든 clock count 단위의 기준입니다.

8. AXI 로 읽을 수 없는 신호와 LED 관측 경로

8.1 AXI 에 없는 두 신호

00 공통명세 9.3 레지스터 맵에 없어서 RTL 도 AXI 로 노출하지 않는 신호가 둘 있습니다.

신호	소스	관측 경로
actuator_enable	safety_controller_core	출력 핀 → led_concat In2 → LED5
control_valid	safety_controller_core	출력 핀 → led_concat In3 → LED6

UART \$MISSION 프레임의 `actuator_enable` / `control_valid` 필드는 하드웨어에서 읽은 값이 아닙니다. 00 공통명세 8.6 정책표에 따라 `system_state` 에서 유도한 값이며 (`MISSION_ActuatorFromState()`), RTL 의 조합 논리를 소프트웨어로 옮긴 것입니다. `SC_ActuatorEnable()` 과 `SC_ControlValid()` 가 같은 식을 구현합니다.

상태	actuator_enable	control_valid
ENABLE=0 또는 fault_valid=0	0	0
NORMAL / WARNING / DEGRADED	1	1
SAFE_MODE	0	0

8.2 Basys 3 LED 맵

led_concat(NUM_PORTS=8) 이 16 비트를 모아 led 포트로 냅니다. 근거 — mission_soc.bd 의 led_concat 파라미터와 net 연결.

LED	led_concat	신호	폭	AXI 로도 읽을 수 있는가
0-1	In0	safety_controller.system_state	2	o SC 0x04
2-4	In1	safety_controller.output_enable	3	o SC 0x08
5	In2	safety_controller.actuator_enable	1	X LED 전용
6	In3	safety_controller.control_valid	1	X LED 전용
7-9	In4	heartbeat_monitor.alive	3	o HB 0x04[2:0]
10-12	In5	heartbeat_monitor.timeout	3	o HB 0x04[10:8]
13-14	In6	fault_manager.fault_level	2	o FM 0x10
15	In7	fault_manager.fault_valid	1	Δ FM CTRL.ENABLE 과 동일

LED5 와 LED6 이 이 설계에서 유일하게 AXI 로 대체할 수 없는 관측점입니다. 보드 검증 시 이 두 개를 반드시 육안으로 확인해야 합니다.

9. 프로그래밍 시퀀스

9.1 부팅 초기화

리셋 직후 모든 IP 는 ENABLE=0 이고, heartbeat_monitor 의 TIMEOUT0/1/2 는 0 입니다. 설정을 먼저 하고 Enable 을 나중에 켜야 합니다. 순서를 바꾸면 Timeout 이 1클럭 만에 성립합니다.

정본은 main.c 의 boot_sequence() 이며, 단계 번호는 04 체크리스트 6장 기준입니다.

1~2.	HB / FM / SC Enable(0) HB / FM / SC EnableIrq(0)	세 IP 를 모두 먼저 끈다
2.5	INJ_ClearAll()	GPIO 주입(error/critical) 제거
3~7.	HB_SetTimeout(0/1/2, clocks) HB_SetAutoRecover(1) FM_SetCriticalMask(0x4) FM_SetPersistLimit(5) SC_SetRecoveryCount(2) SC_SetDegradeMask(0x1)	300 / 600 / 150 ms
8.	HB_ClearAll() + HB_ClearIrq(0x7) FM_ResetFault() + FM_ClearIrq(0x1) SC_ClearIrq(0x1)	
8.5	HBGEN_Init()	
9.	FM_SelfCheck()	FM_ID(0x2C) == 0x464D4752 확인
10.	MissionIntr_Init()	AXI INTC 초기화 / Handler 등록
11.	FM_Enable(1) → SC_Enable(1)	
12.	HB_Enable(1)	
13.	FM_EnableIrq(1) → HB_EnableIrq(1) → SC_EnableIrq(1)	
14.	Xil_ExceptionEnable()	MicroBlaze Global Interrupt

각 단계에는 이유가 있으므로 임의로 바꾸지 마십시오.

단계	이 순서인 이유
1~2 를 IP 별이 아니라 전역으로 묶은 이유	Cold Reset 이면 세 IP 가 전부 Disable 이라 상관없지만, 디버거 CPU Reset 처럼 MicroBlaze 만 재시작하고 AXI 주변 IP 는 리셋 되지 않는 경우가 있습니다. HB 를 재설정하는 동안 FM 과 SC 가 Enable 인 채로 HB 의 과도기 출력을 읽으면 엉뚱한 Fault / State 이벤트가 순간적으로 발생합니다
2.5 INJ_ClearAll() 이 설정보다 먼저인 이유	GPIO 로 주입한 error_flag 와 critical_fault 는 CPU Reset 으로 지워지지 않습니다. 8단계의 FM_ResetFault() 가 성립하려면(device_fault == 0) 여기서 먼저 지워야 합니다. 4.3 절의 조건부 명령을 참고하십시오
HB_SetAutoRecover(1) 인 이유	0 이면 Timeout 래치가 CLEAR_ALL 전까지 풀리지 않아, Heartbeat 를 다시 보내도 timeout 이 살아 있습니다. "모든 Fault 제거 → Level 0 → NORMAL" 이 성립하지 않습니다

단계	이 순서인 이유
11 에서 FM 이 SC 보다 먼저인 이유	SC 는 <code>fault_valid=0</code> 이면 출력을 안전값으로 강제합니다. <code>fault_valid</code> 는 FM 의 <code>ENABLE</code> 이므로 FM 을 먼저 켜야 SC 가 정상 상태로 출발합니다
12 에서 HB 가 마지막인 이유	HB 를 켜는 순간부터 Timeout Counter 가 돌기 시작합니다. FM 과 SC 가 이미 준비된 뒤여야 첫 Timeout 을 정상 경로로 처리합니다
13 이 Enable 보다 뒤인 이유	IP 를 켜는 과도기에 IRQ 가 뜨는 것을 막습니다

`RECOVERY_COUNT < PERSIST_LIMIT` 을 지키십시오. 어기면 하강 전이가 성립하지 않습니다.

`mission_ip_regs.h` 에 선언된 `HB_Init()` / `FM_Init()` / `SC_Init()` 는 IP 하나만 다루는 구형 진입점입니다. `boot_sequence()` 는 이들을 사용하지 않고 개별 setter 를 전역 단계로 재배치해 호출합니다. 위 표의 첫 줄이 그 이유입니다.

9.2 CTRL Shadow

2.3 절에서 설명한 대로 W1P 명령은 반드시 Shadow 와 OR 해서 보냅니다. `fm_regs.c` / `hb_regs.c` / `sc_regs.c` 가 각각 정적 Shadow 변수를 유지합니다.

```
/* fm_regs.c */
static u32 s_fm_ctrl;                /* ENABLE 등 RW 비트만 보존 */

void FM_Enable(int on)
{
    if (on) s_fm_ctrl |=  FM_CTRL_ENABLE;
    else    s_fm_ctrl &= ~FM_CTRL_ENABLE;
    REG_WR(FM_BASE, FM_CTRL, s_fm_ctrl);
}

void FM_ResetFault(void)             /* W1P */
{
    REG_WR(FM_BASE, FM_CTRL, s_fm_ctrl | FM_CTRL_RESET_FAULT);
}
```

`heartbeat_monitor` 는 `ENABLE`(bit0) 과 `AUTO_RECOVER`(bit2) 를 모두 Shadow 에 넣어야 합니다. `FM_IsEnabled()` 와 `SC_IsEnabled()` 는 하드웨어를 읽지 않고 이 Shadow 를 반환합니다.

9.3 조건부 명령의 결과 확인

`SLVERR` 가 없으므로 거부된 명령도 `OKAY` 를 반환합니다. 다음 두 명령은 결과를 다시 읽어 확인해야 합니다.

명령	성립 조건	확인 방법
<code>FM_CTRL.RESET_FAULT</code>	<code>FAULT_INPUT</code> 의 세 필드 OR 가 0	<code>FAULT_LEVEL</code> 과 <code>FAULT_COUNT</code> 재확인
<code>SC_CTRL.MANUAL_RESET</code>	<code>SYSTEM_STATE==3 && fault_valid && FAULT_LEVEL==0</code>	<code>SYSTEM_STATE</code> 재확인

9.4 ISR

1. IRQ_STATUS 읽기

(FM/HB 0x24, SC 0x1C)
2. 읽은 값 그대로 되쓰기 (W1C)
3. 상태 레지스터 스냅샷 저장
4. 무거운 처리는 메인 루프로

`heartbeat_monitor` 는 `CTRL.CLEAR_ALL` 로 Pending 이 지워지지 않습니다. 3.3 절을 참고하십시오.

10. 알려진 제약과 주의사항

#	항목	영향	대응
1	세 IP 모두 <code>SLVERR</code> 와 <code>DECERR</code> 를 내지 않습니다	잘못된 주소 접근이 조용히 0 을 반환하거나 버려집니다	<code>FM_ID</code> 로 주소 매핑을 검증합니다
2	Aperture 64 KB 대비 디코딩 폭이 좁습니다	64 B(SC 는 32 B) 주기로 레지스터가 반복 노출됩니다	매크로만 사용하고 오프셋을 직접 쓰지 않습니다
3	SC 의 0x20 은 CTRL 의 에일리어스입니다	FM/HB 습관대로 0x20 에 <code>IRQ_EN</code> 을 쓰면 SC 가 꺼 집니다	<code>SC_IRQ_EN</code> 매크로(0x18)를 사용합니다
4	비정렬 접근 동작이 IP 마다 다릅니다	FM 은 하위 2비트를 무시하고, HB 와 SC 는 0 을 반환 합니다	32 bit 정렬 접근만 사용합니다
5	FM 은 <code>WSTRB[0]==0</code> Write 를 통째로 무시합니다	Byte1~3 만 쓰는 접근이 무효가 됩니다	32 bit 단위로 씁니다

#	항목	영향	대응
6	CTRL Write 가 ENABLE 을 항상 덮어씁니다	W1P 단독 Write 시 IP 가 꺼집니다	CTRL Shadow 를 사용합니다 (2.3, 9.2)
7	HB TIMEOUTn 리셋값이 0(유효값 1)입니다	설정 전에 Enable 하면 즉시 Timeout 이 됩니다	설정 후 Enable 합니다 (9.1)
8	HB CTRL.CLEAR_ALL 이 IRQ_STATUS 를 지우지 않습니다	Pending 이 남아 IRQ 가 계속 High 를 유지합니다	IRQ_STATUS 에 별도로 W1C 합니다
9	FM FAULT_COUNT 가 0xFF 에서 포화합니다	PERSIST_LIMIT=255 면 DEGRADED-WARNING 하강이 불가능합니다	운용값 5 를 사용합니다
10	SC STATE_TIMER 가 0xFFFF_FFFF 에서 포화합니다	약 42.9 초 이후 값이 멈춥니다	장기 유지 시간은 소프트웨어가 셉니다
11	actuator_enable 과 control_valid 를 AXI 로 읽을 수 없습니다	UART 필드는 유도값입니다	8장을 참고하고 LED 로 실측합니다
12	폐기된 HDL 파일이 리포지토리에 남아 있습니다	사양을 잘못 읽을 수 있습니다	10.1 절을 참고하고 component.xml fileSet 을 기준으로 삼습니다

10.1 폐기 및 미사용 HDL 파일

파일	상태
SOC_Pr/ip_repo/myip_heartbeat_monitor_1_0/hdl/myip_heartbeat_monitor_slave_lite_v1_0_S00_AXI.v	component.xml fileSet 에 없습니다. 합성 대상이 아니며 사양 근거로 사용하면 안 됩니다
rtl/fault_manager_ip_v1_0.v, rtl/fault_manager_ip_v1_0_S00_AXI.v	IP 패키징 이전의 초기 래퍼입니다. 현재 IP 는 hdl/fault_manager_ip.v 를 사용합니다

rtl/ 의 **fault_manager_axi.v, fault_manager_core.v, eval_tick_generator.v** 는 ip_repo/ 및 Block Design 의 **ipshared** 사본과 바이트 단위로 동일합니다. 어느 쪽을 참고해도 결과가 같습니다.

11. 사양 근거 파일

대상	정본 파일
heartbeat_monitor AXI + Core	SOC_Pr/ip_repo/myip_heartbeat_monitor_1_0/src/heartbeat_monitor.v
heartbeat_monitor Top	SOC_Pr/ip_repo/myip_heartbeat_monitor_1_0/hdl/myip_heartbeat_monitor.v
fault_manager AXI	SOC_Pr/ip_repo/fault_manager_ip_1_0/src/fault_manager_axi.v
fault_manager Core	SOC_Pr/ip_repo/fault_manager_ip_1_0/src/fault_manager_core.v
fault_manager Top	SOC_Pr/ip_repo/fault_manager_ip_1_0/hdl/fault_manager_ip.v
safety_controller AXI (Top)	SOC_Pr/ip_repo/safety_controller_1_0/hdl/safety_controller_slave_lite_v1_0_S00_AXI.v
safety_controller Core	SOC_Pr/ip_repo/safety_controller_1_0/hdl/safety_controller.v
eval_tick_generator	rtl/eval_tick_generator.v
주소 맵 · IRQ 배선 · IP 파라미터	SOC_Pr/soc_project/soc_project.srcs/sources_1/bd/mission_soc/mission_soc.bd
소프트웨어 레지스터 정의	SOC_Pr_Vitis/soc_prj/src/mission_ip_regs.h
인터럽트 ID	SOC_Pr_Vitis/soc_prj/src/mission_intr.h
상위 명세	00_TEAM_COMMON_SPEC_3MEMBERS_RECOVERY_FIXED.md 7장 · 9장 · 10장